Sesión 22: Consumo de APIs REST (Cliente HTTP con Dio, Manejo de Interceptores, Encabezados y Respuestas Asíncronas)

Introducción

En aplicaciones móviles de nivel profesional, la interacción con servicios web externos y microservicios es un componente crítico. Si bien el paquete nativo `http` de Dart ofrece capacidades básicas para solicitudes de red, aplicaciones con requerimientos complejos demandan características avanzadas como configuración global de tiempos de espera, inyección de encabezados de autenticación, transformaciones automáticas de datos y reintentos ante fallos de conexión.

En esta sesión abordaremos el consumo de APIs REST utilizando **Dio**, el cliente HTTP potente y flexible para Dart/Flutter. Estudiaremos la configuración centralizada de `BaseOptions`, la implementación de interceptores (`InterceptorsWrapper`) para auditar peticiones y manejar tokens de sesión, la gestión de excepciones estructuradas mediante `DioException` y el procesamiento asíncrono de respuestas en la capa de infraestructura.

Conceptos y Definiciones

El paquete `dio` simplifica la capa de red al proporcionar una abstracción robusta alrededor de las solicitudes HTTP, permitiendo intercepte, modifique y rastree cada petición y respuesta de forma centralizada.



```
HTTP/Red.|
| • headers de     |            | • onResponse:    |            | •
Clasificación: |
|   aplicación.    |              Logger/Log de    |              badResponse,
|
|                  |              respuestas.      |
connectionTimeout|
|                  |            | • onError:       |
connectionError. |
|                  |              Reintentos o     |
|
|                  |              refresh token.   |
|
+------------------+            +------------------+            +---------------
---+
```

1. Configuración Centralizada con `BaseOptions`

- **`BaseOptions`:** Permite definir la configuración por defecto para todas las peticiones realizadas por la instancia del cliente `Dio`.
- **Propiedades Clave:**
- `baseUrl`: La URL raíz de la API REST (ej. `[https://api.escom.ipn.mx/v1](https://api.escom.ipn.mx/v1)`).
- `connectTimeout`: Tiempo límite para establecer la conexión con el servidor remoto.
- `receiveTimeout`: Tiempo límite para recibir la transmisión de datos desde la red.
- `headers`: Encabezados HTTP predeterminados (ej. `Content-Type: application/json`, `Accept: application/json`).

---

2. Ciclo de Vida de los Interceptores (`Interceptors`)

Los interceptores funcionan como middlewares en la capa de red, permitiendo inspeccionar o transformar las peticiones antes de que salgan del cliente o antes de que las respuestas lleguen a la lógica de la aplicación.

- **`onRequest`:** Se ejecuta antes de enviar la solicitud. Ideal para inyectar automáticamente encabezados de autenticación (`Authorization: Bearer <TOKEN>`) o parámetros globales de auditoría.
- **`onResponse`:** Se ejecuta cuando se recibe una respuesta exitosa (código de estado `2xx`). Permite implementar registros (*logging*) centralizados o des-serialización común.
- **`onError`:** Captura las respuestas no exitosas o fallos de red. Permite implementar lógica de refresco automático de token (*Refresh Token*) o conversión a errores de dominio.

---

3. Control de Errores con `DioException`

A diferencia del paquete `http` estándar, `Dio` lanza excepciones explícitas de tipo `DioException` cuando el código de respuesta HTTP no pertenece al rango `2xx` o cuando ocurre un fallo de conectividad.

- **`DioExceptionType.connectionTimeout`:** Excesivo tiempo consumido tratando de conectar al servidor.

- `DioExceptionType.badResponse`: El servidor respondió con un estado de error (401, 403, 404, 500).
- `DioExceptionType.connectionError`: Sin conectividad de red o problemas de resolución DNS.

Ejemplo Completo con Código Fuente: Cliente de Red Profesional y Servicio REST de Cursos ESCOM

El siguiente ejemplo implementa un cliente HTTP reutilizable utilizando `Dio` estructurado con `BaseOptions`, un interceptor de logs/autenticación, y una interfaz Flutter que consume asíncronamente un listado de unidades de aprendizaje.

> **Nota de dependencia:** Para ejecutar este ejemplo se requiere incluir `dio: ^5.4.3+1` en el archivo `pubspec.yaml`.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:dio/dio.dart';

void main() {
  runApp(const DemoDioApp());
}

// ============================================================================
// MODELO DE DATOS (DTO / ENTIDAD)
// ============================================================================

class UnidadAprendizaje {
  final int id;
  final String nombre;
  final String codigo;
  final int creditos;

  const UnidadAprendizaje({
    required this.id,
    required this.nombre,
    required this.codigo,
    required this.creditos,
  });

  factory UnidadAprendizaje.fromJson(Map<String, dynamic> json) {
    return UnidadAprendizaje(
      id: json['id'] ?? 0,
      nombre: json['title'] ?? 'Sin Nombre', // Se mapea 'title' del API pública
      codigo: 'UA-${json['id']}',
      creditos: (json['userId'] as int? ?? 1) * 3, // Cálculo simulado
    );
  }
}

// ============================================================================
// CLIENTE HTTP CON DIO Y CONFIGURACIÓN DE INTERCEPTORES
```

```dart
// ================================================================================

class ApiClient {
  late final Dio _dio;

  ApiClient() {
    // 1. BASEOPTIONS: Configuración global del cliente
    _dio = Dio(
      BaseOptions(
        baseUrl: 'https://jsonplaceholder.typicode.com', // API REST de prueba
pública
        connectTimeout: const Duration(seconds: 5),
        receiveTimeout: const Duration(seconds: 5),
        headers: {
          'Content-Type': 'application/json',
          'Accept': 'application/json',
          'X-Client-Platform': 'Flutter-ESCOM',
        },
      ),
    );

    // 2. INTERCEPTORES: Middleware para Auth y Logging
    _dio.interceptors.add(
      InterceptorsWrapper(
        onRequest: (options, handler) {
          // Inyección simulada de Token JWT
          const token = 'JWT_TOKEN_EJEMPLO_ESCOM_2026';
          options.headers['Authorization'] = 'Bearer $token';

          debugPrint('--> HTTP ${options.method} ${options.uri}');
          debugPrint('Headers: ${options.headers}');
          return handler.next(options); // Continuar con la petición
        },
        onResponse: (response, handler) {
          debugPrint('<-- HTTP ${response.statusCode}
[${response.requestOptions.path}]');
          return handler.next(response); // Continuar con la respuesta
        },
        onError: (DioException error, handler) {
          debugPrint('!!! ERROR HTTP: [${error.type}] ${error.message}');
          return handler.next(error); // Continuar con el error
        },
      ),
    );
  }

  // MÉTODO ASÍNCRONO PARA OBTENER RECURSOS (GET)
  Future<List<UnidadAprendizaje>> obtenerUnidadesAprendizaje() async {
    try {
      final response = await _dio.get('/posts');

      if (response.statusCode == 200) {
        final List<dynamic> listJson = response.data;
        // Mapeo asíncrono y transformación de JSON a objetos
```

```
        return listJson
            .take(10) // Tomamos solo los primeros 10 elementos
            .map((json) => UnidadAprendizaje.fromJson(json))
            .toList();
      } else {
        throw Exception('Respuesta inesperada del servidor:
${response.statusCode}');
      }
    } on DioException catch (e) {
      // Manejo estructurado de errores específicos de Dio
      throw _manejarErrorDio(e);
    }
  }

  String _manejarErrorDio(DioException error) {
    switch (error.type) {
      case DioExceptionType.connectionTimeout:
        return 'Tiempo de espera agotado al conectar con el servidor.';
      case DioExceptionType.receiveTimeout:
        return 'El servidor tardó demasiado en responder.';
      case DioExceptionType.badResponse:
        return 'Error en el servidor: Código ${error.response?.statusCode}.';
      case DioExceptionType.connectionError:
        return 'Sin conexión a red. Verifique su acceso a Internet.';
      default:
        return 'Error de red no identificado: ${error.message}';
    }
  }
}

// ================================================================================
// INTERFAZ DE USUARIO (PRESENTATION)
// ================================================================================

class DemoDioApp extends StatelessWidget {
  const DemoDioApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Consumo API REST con Dio',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaCatalogoCursos(),
    );
  }
}

class PantallaCatalogoCursos extends StatefulWidget {
```

```
  const PantallaCatalogoCursos({super.key});

  @override
  State<PantallaCatalogoCursos> createState() => _PantallaCatalogoCursosState();
}

class _PantallaCatalogoCursosState extends State<PantallaCatalogoCursos> {
  final ApiClient _apiClient = ApiClient();
  late Future<List<UnidadAprendizaje>> _futureUnidades;

  @override
  void initState() {
    super.initState();
    _futureUnidades = _apiClient.obtenerUnidadesAprendizaje();
  }

  void _recargarDatos() {
    setState(() {
      _futureUnidades = _apiClient.obtenerUnidadesAprendizaje();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catálogo de Materias (Dio)'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: _recargarDatos,
            tooltip: 'Reintentar petición',
          ),
        ],
      ),
      body: FutureBuilder<List<UnidadAprendizaje>>(
        future: _futureUnidades,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  CircularProgressIndicator(),
                  SizedBox(height: 12),
                  Text('Conectando con la API REST via Dio...'),
                ],
              ),
            );
          }

          if (snapshot.hasError) {
```

```
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24.0),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    const Icon(Icons.wifi_off, color: Colors.red, size: 60),
                    const SizedBox(height: 12),
                    Text(
                      'Error al consultar API',
                      style: Theme.of(context).textTheme.titleLarge,
                    ),
                    const SizedBox(height: 8),
                    Text(
                      snapshot.error.toString(),
                      textAlign: TextAlign.center,
                      style: const TextStyle(color: Colors.grey),
                    ),
                    const SizedBox(height: 16),
                    ElevatedButton(
                      onPressed: _recargarDatos,
                      child: const Text('Reintentar Petición'),
                    ),
                  ],
                ),
              ),
            );
          }

          if (!snapshot.hasData || snapshot.data!.isEmpty) {
            return const Center(child: Text('No hay materias disponibles.'));
          }

          final unidades = snapshot.data!;

          return ListView.builder(
            padding: const EdgeInsets.all(12),
            itemCount: unidades.length,
            itemBuilder: (context, index) {
              final ua = unidades[index];
              return Card(
                elevation: 2,
                margin: const EdgeInsets.only(bottom: 10),
                child: ListTile(
                  leading: CircleAvatar(
                    backgroundColor:
Theme.of(context).colorScheme.primaryContainer,
                    child: Text('${ua.id}'),
                  ),
                  title: Text(
                    ua.nombre,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(fontWeight: FontWeight.bold),
```

```
                    ),
                    subtitle: Text('Código: ${ua.codigo}'),
                    trailing: Chip(
                      label: Text('${ua.creditos} SATCA'),
                    ),
                  ),
                );
              },
            );
          },
        ),
      );
    }
  }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Implementación de Peticiones POST y Cancelación con `CancelToken`

1. Añade un método `Future<void> crearUnidad(UnidadAprendizaje nuevaUa, CancelToken cancelToken)` en la clase `ApiClient` para realizar solicitudes `POST` usando `_dio.post('/posts', data: ...)`.
2. Asocia un `CancelToken` a la solicitud y agrega un botón en la interfaz de usuario que permita cancelar manualmente la petición HTTP en progreso invocado `cancelToken.cancel('Operación cancelada por el usuario')`.
3. Maneja la excepción `DioExceptionType.cancel` para notificar al usuario en pantalla.

Práctica 2: Interceptor Avanzado para Manejo de Expiración de Token (401 Unauthorized)

1. Simula un error `401 Unauthorized` modificando la respuesta del interceptor `onError`.
2. Implementa dentro de `onError` una verificación: si `error.response?.statusCode == 401`, ejecuta una función de re-autenticación asíncrona (*refresh token*), actualiza el encabezado `Authorization` con el nuevo token y re-intenta la petición original usando `_dio.fetch(error.requestOptions)`.

Resumen y Puntos Clave

- **Dio vs HTTP Nativo:** Dio proporciona una solución integral para aplicaciones enterprise, ofreciendo `BaseOptions`, cancelaciones, descargas en segundo plano e interceptores.
- **Centralización con `BaseOptions`:** Define URLs base, timeouts y encabezados globales en un único punto del proyecto.
- **Interceptores como Middleware:** Permiten aislar la lógica de autenticación (tokens Bearer), auditoría de logs y manejo global de refrescos de sesión.
- **Manejo Estructurado de `DioException`:** Facilita la identificación precisa de fallos de red, timeouts o respuestas erróneas del servidor para ofrecer retroalimentación clara en la interfaz.

Bibliografía y Referencias

1. **Flutter China & Open Source Contributors.** (2024). *Dio: A powerful HTTP client for Dart/Flutter*. Recuperado de: https://pub.dev/packages/dio
2. **Google LLC.** (2024). *Fetch data from the internet in Flutter*. Recuperado de: https://docs.flutter.dev/cookbook/networking/fetch-data
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.